

Fundamentals and the Singular Value Decomposition

MATH 616: Computational Linear Algebra

Dr. Xuping Xie

Department of Mathematics & Statistics
Old Dominion University

August 27, 2026

Outline

- 1 Course Overview
- 2 Review: Norms
- 3 Review: Orthogonality and Projectors
- 4 The Singular Value Decomposition

Course Overview

What This Course Is About

Numerical linear algebra is the study of **algorithms** for the two central problems of matrix computation:

- Solve $A\mathbf{x} = \mathbf{b}$
- Find eigenvalues/eigenvectors: $A\mathbf{v} = \lambda\mathbf{v}$

Three recurring questions for every algorithm:

- 1 Does it give the right answer? (**stability**)
- 2 How sensitive is the problem itself? (**conditioning**)
- 3 How fast is it? (**complexity, structure, sparsity**)

Two running application threads:

- **Machine learning**: PCA, regression, spectral methods, and research spotlights (LoRA, deep equilibrium models, learned preconditioners)
- **Scientific computing**: where the large sparse systems come from

The Central Object of Weeks 1–2: the SVD

Every matrix $A \in \mathbb{R}^{m \times n}$ can be written as

$$A = U\Sigma V^T \tag{1}$$

with U, V orthogonal and Σ diagonal with nonnegative entries.

Why start here?

- It answers “what does a matrix *do*?” completely
- It defines rank, norms, condition numbers, pseudoinverses
- It is the optimal tool for **low-rank approximation** — the mathematical core of PCA, image compression, recommender systems, and LoRA fine-tuning of large language models

Motivation

You already “know” how to do everything in this course:

- Solve $A\mathbf{x} = \mathbf{b}$: $\mathbf{x} = A^{-1}\mathbf{b}$
- Eigenvalues: roots of $\det(A - \lambda I) = 0$
- Least squares: $\mathbf{x} = (A^T A)^{-1} A^T \mathbf{b}$

Why each one fails in practice:

- **Cramer’s rule** via cofactors costs $O(n!)$. For $n = 25$: longer than the age of the universe. Gaussian elimination: microseconds.
- **Forming** A^{-1} costs $3\times$ more than solving the system and is *less accurate*. Rule of this course: **never compute an inverse to solve a system**.
- **The characteristic polynomial**: roots are hypersensitive to coefficient errors.
- $(A^T A)^{-1} A^T$ squares the sensitivity of the problem.

The gap between “mathematically correct” and “computationally viable” is this entire course.

Motivation: How Do We Even Talk About Error?

A scenario you will face constantly. Two algorithms return two candidate solutions to the same system $A\mathbf{x} = \mathbf{b}$:

$$\hat{\mathbf{x}}_1 \text{ with residual } \|\mathbf{b} - A\hat{\mathbf{x}}_1\| = 10^{-13}, \quad \hat{\mathbf{x}}_2 \text{ with residual } \|\mathbf{b} - A\hat{\mathbf{x}}_2\| = 10^{-5} \quad (2)$$

Is $\hat{\mathbf{x}}_1$ the better answer?

Not necessarily. A small *residual* (equation nearly satisfied) does not imply a small *error* $\|\hat{\mathbf{x}} - \mathbf{x}_{\text{true}}\|$ — for ill-conditioned A they can differ by many orders of magnitude.

Distinguishing these two notions is a central theme (Week 6), and both require a way to measure the “size” of vectors and matrices.

Norms are the measuring instruments of numerical mathematics. Every statement we will make this semester — “the error is small,” “the algorithm is stable,” “the approximation is optimal” — is a statement about a norm. Different norms formalize different questions:

- worst single entry off? (ℓ_∞) total budget of deviations? (ℓ_1) energy? (ℓ_2)

So we begin by building the instruments.

Review: Norms

Vector Norms

A **norm** $\|\cdot\| : \mathbb{R}^n \rightarrow \mathbb{R}$ satisfies, for all $\mathbf{x}, \mathbf{y}$ and scalars α :

- ① $\|\mathbf{x}\| \geq 0$, with equality iff $\mathbf{x} = \mathbf{0}$
- ② $\|\alpha\mathbf{x}\| = |\alpha| \|\mathbf{x}\|$
- ③ $\|\mathbf{x} + \mathbf{y}\| \leq \|\mathbf{x}\| + \|\mathbf{y}\|$ (triangle inequality)

The p -norms:

$$\|\mathbf{x}\|_p = \left(\sum_{i=1}^n |x_i|^p \right)^{1/p}, \quad \|\mathbf{x}\|_\infty = \max_i |x_i| \quad (3)$$

Most important cases: $\|\mathbf{x}\|_1$ (sum of magnitudes), $\|\mathbf{x}\|_2$ (Euclidean), $\|\mathbf{x}\|_\infty$ (max).

ML aside: the choice of norm is modeling. ℓ_2 penalties give ridge regression; ℓ_1 gives lasso and sparsity.

Example: Computing Vector Norms

Take $\mathbf{x} = (1, -2, 2)^T$.

$$\|\mathbf{x}\|_1 = |1| + |-2| + |2| = 5 \quad (4)$$

$$\|\mathbf{x}\|_2 = \sqrt{1 + 4 + 4} = 3 \quad (5)$$

$$\|\mathbf{x}\|_\infty = \max\{1, 2, 2\} = 2 \quad (6)$$

Always true: $\|\mathbf{x}\|_\infty \leq \|\mathbf{x}\|_2 \leq \|\mathbf{x}\|_1 \leq n\|\mathbf{x}\|_\infty$ (here: $2 \leq 3 \leq 5 \leq 6 \checkmark$)

All norms on $\mathbb{R}^n$ are equivalent — they differ by constants depending on n . For convergence theory the choice is irrelevant; **for algorithm design and error measurement the constants matter**: with $n = 10^6$ pixels, $\|\cdot\|_1$ and $\|\cdot\|_\infty$ can disagree by a factor of a million.

Exercise (30 seconds): for which vectors is $\|\mathbf{x}\|_1 = \|\mathbf{x}\|_2 = \|\mathbf{x}\|_\infty$?

Inner Products and the Cauchy–Schwarz Inequality

The Euclidean inner product:

$$\langle \mathbf{x}, \mathbf{y} \rangle = \mathbf{x}^T \mathbf{y} = \sum_{i=1}^n x_i y_i, \quad \|\mathbf{x}\|_2 = \sqrt{\mathbf{x}^T \mathbf{x}} \quad (7)$$

Cauchy–Schwarz:

$$|\mathbf{x}^T \mathbf{y}| \leq \|\mathbf{x}\|_2 \|\mathbf{y}\|_2 \quad (8)$$

with equality iff $\mathbf{x}, \mathbf{y}$ are parallel.

Defines angles: $\cos \theta = \frac{\mathbf{x}^T \mathbf{y}}{\|\mathbf{x}\|_2 \|\mathbf{y}\|_2}$.

This quantity is “cosine similarity” — the standard measure of closeness for embedding vectors in machine learning.

Matrix Norms

Induced (operator) norms measure how much A can stretch a vector:

$$\|A\|_p = \max_{\mathbf{x} \neq \mathbf{0}} \frac{\|A\mathbf{x}\|_p}{\|\mathbf{x}\|_p} \quad (9)$$

Special cases:

- $\|A\|_1 = \max$ absolute column sum
- $\|A\|_\infty = \max$ absolute row sum
- $\|A\|_2 = \sigma_1(A)$, the largest singular value

Frobenius norm (treat A as a long vector):

$$\|A\|_F = \left(\sum_{i,j} |a_{ij}|^2 \right)^{1/2} = \sqrt{\text{tr}(A^T A)} \quad (10)$$

Key properties: submultiplicativity $\|AB\| \leq \|A\| \|B\|$; invariance of $\|\cdot\|_2$ and $\|\cdot\|_F$ under orthogonal transformations.

Example: Matrix Norms

Meet the matrix we will follow all week:

$$A = \begin{bmatrix} 3 & 0 \\ 4 & 5 \end{bmatrix} \quad (11)$$

$$\|A\|_1 = \max\{|3| + |4|, |0| + |5|\} = \max\{7, 5\} = 7 \quad (\text{column sums}) \quad (12)$$

$$\|A\|_\infty = \max\{|3| + |0|, |4| + |5|\} = \max\{3, 9\} = 9 \quad (\text{row sums}) \quad (13)$$

$$\|A\|_F = \sqrt{9 + 0 + 16 + 25} = \sqrt{50} \approx 7.07 \quad (14)$$

And the 2-norm? $\|A\|_2 = \max_{\|x\|_2=1} \|Ax\|_2$ has no entrywise formula.

Try a few unit vectors: $\|Ae_1\|_2 = \|(3, 4)\|_2 = 5$, $\|Ae_2\|_2 = \|(0, 5)\|_2 = 5$. Is the max 5?

No: try $x = \frac{1}{\sqrt{2}}(1, 1)^T$: $\|Ax\|_2 = \frac{1}{\sqrt{2}}\|(3, 9)\|_2 = \sqrt{45} \approx 6.71 > 5$.

We will prove $\|A\|_2 = \sigma_1 = \sqrt{45}$ exactly — and that this x is the maximizer.

Review: Orthogonality and Projectors

Motivation: The Right Basis Makes Hard Problems Easy

Almost every algorithm in this course has the same secret structure:

change to a well-chosen basis $\rightarrow$ solve a trivial problem there $\rightarrow$ change back

Examples we will meet:

- SVD: in the right pair of bases, every matrix is diagonal
- Least squares: in a basis adapted to $\text{range}(A)$, fitting = “keep some coordinates, discard the rest”
- PCA: in the principal basis, data coordinates are uncorrelated and ordered by importance

But a change of basis can also destroy a computation: if the new basis vectors are nearly parallel, coordinates involve huge cancellations — tiny data errors explode.

Orthonormal bases are the only ones that are always safe: coordinates are inner products, lengths are preserved, rounding errors pass through unamplified. That is why the factorizations of numerical linear algebra (SVD, QR, Schur) are built exclusively from orthogonal transformations — and why we review them now.

Orthogonal Vectors and Matrices

Vectors $\mathbf{q}_1, \dots, \mathbf{q}_k$ are **orthonormal** if

$$\mathbf{q}_i^T \mathbf{q}_j = \delta_{ij} \quad (15)$$

A square matrix $Q \in \mathbb{R}^{n \times n}$ is **orthogonal** if $Q^T Q = Q Q^T = I$, i.e., its columns form an orthonormal basis.

Why we love orthogonal matrices:

- They preserve lengths: $\|Q\mathbf{x}\|_2 = \|\mathbf{x}\|_2$
- They preserve angles: $(Q\mathbf{x})^T(Q\mathbf{y}) = \mathbf{x}^T\mathbf{y}$
- $Q^{-1} = Q^T$ — inversion is free
- **They do not amplify rounding errors** — the reason numerically stable algorithms (Householder QR, the SVD) are built from them

Examples: The Orthogonal Matrix Zoo

Rotation (determinant $+1$):

$$R_\theta = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}, \quad R_\theta^T R_\theta = I \quad \checkmark \quad (16)$$

Reflection (determinant -1): with unit vector $\mathbf{q} = \frac{1}{\sqrt{2}}(1, 1)^T$,

$$F = I - 2\mathbf{q}\mathbf{q}^T = \begin{bmatrix} 0 & -1 \\ -1 & 0 \end{bmatrix} \quad (17)$$

Check on $\mathbf{x} = (3, 1)^T$: $F\mathbf{x} = (-1, -3)^T$ — same length ($\sqrt{10}$), reflected. This construction, a **Householder reflector**, is the engine of QR factorization (Week 4).

Permutation: reordering rows/columns; $P^T P = I$ trivially. Central to pivoting.

Fact: every 2×2 orthogonal matrix is a rotation or a reflection. In n dimensions, orthogonal matrices are products of reflections — one reason Householder reflectors suffice to build any Q .

Projectors

A matrix $P \in \mathbb{R}^{n \times n}$ is a **projector** if

$$P^2 = P \quad (18)$$

It is an **orthogonal projector** if additionally $P = P^T$.

Geometry: P maps every vector onto $\text{range}(P)$; an orthogonal projector does so along the perpendicular direction. The complement $I - P$ projects onto the orthogonal subspace.

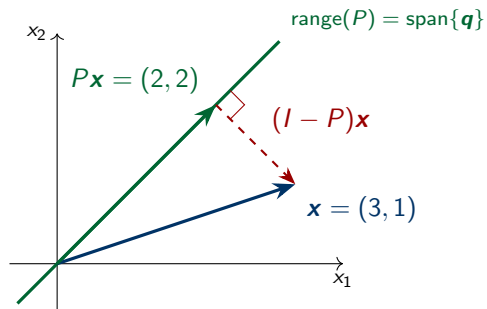
Construction. If $Q = [\mathbf{q}_1, \dots, \mathbf{q}_k]$ has orthonormal columns,

$$P = QQ^T = \sum_{i=1}^k \mathbf{q}_i \mathbf{q}_i^T \quad (19)$$

projects orthogonally onto $\text{span}\{\mathbf{q}_1, \dots, \mathbf{q}_k\}$.

Preview: least squares fitting (Week 5) is orthogonal projection of $\mathbf{b}$ onto $\text{range}(A)$; PCA (Week 2) projects data onto the span of top singular vectors.

Geometry of Orthogonal Projection



How to read the picture:

- $P\mathbf{x}$ is the **shadow** of $\mathbf{x}$ on the subspace, cast at noon (perpendicular light)
- the **residual** $(I - P)\mathbf{x} \perp \text{range}(P)$: *orthogonality of the error is the signature of orthogonal projection*
- $P\mathbf{x}$ is the **closest point** of the subspace to $\mathbf{x}$ — tilting the dashed segment away from perpendicular only makes it longer
- applying P again moves nothing: the shadow of a shadow is itself ($P^2 = P$)

“Closest point in a subspace” is precisely what least squares asks for — this picture is Week 5, one slide early.

Example: A Projector in Action

Project onto the line spanned by $\mathbf{q} = \frac{1}{\sqrt{2}}(1, 1)^T$: $P = \mathbf{q}\mathbf{q}^T = \frac{1}{2} \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$

Verify the axioms:

$$P^2 = \frac{1}{4} \begin{bmatrix} 2 & 2 \\ 2 & 2 \end{bmatrix} = P \quad \checkmark \quad P^T = P \quad \checkmark \quad (20)$$

Apply to $\mathbf{x} = (3, 1)^T$:

$$P\mathbf{x} = \begin{bmatrix} 2 \\ 2 \end{bmatrix} \quad (\text{the component along the line}), \quad (I-P)\mathbf{x} = \begin{bmatrix} 1 \\ -1 \end{bmatrix} \quad (\text{the perpendicular residual}) \quad (21)$$

Checks: $P\mathbf{x} + (I-P)\mathbf{x} = \mathbf{x}$, $(2, 2) \perp (1, -1)$, and Pythagoras:

$$\|\mathbf{x}\|_2^2 = 10 = \|P\mathbf{x}\|_2^2 + \|(I-P)\mathbf{x}\|_2^2 = 8 + 2 \quad \checkmark$$

In Week 5, “fit a model by least squares” will mean exactly: compute $P\mathbf{b}$ for the right subspace. The residual $(I-P)\mathbf{b}$ is what your model cannot explain.

Rank, Range, and Null Space

For $A \in \mathbb{R}^{m \times n}$:

- $\text{range}(A) = \{A\mathbf{x} : \mathbf{x} \in \mathbb{R}^n\} \subseteq \mathbb{R}^m$
- $\text{null}(A) = \{\mathbf{x} : A\mathbf{x} = \mathbf{0}\} \subseteq \mathbb{R}^n$
- $\text{rank}(A) = \dim \text{range}(A)$

Fundamental theorem: $\text{rank}(A) + \dim \text{null}(A) = n$, and

$$\text{null}(A) = \text{range}(A^T)^\perp \quad (22)$$

Question for Thursday: in floating point arithmetic, the rank of a stored matrix is almost always $\min(m, n)$ — tiny perturbations destroy exact rank deficiency. What is a *useful* notion of rank for computed data?

The answer requires singular values.

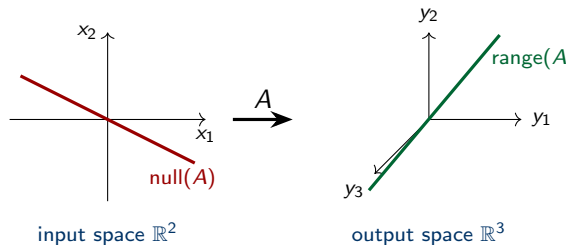
Example: Rank, Range, and Null Space, Concretely

$$A = \begin{bmatrix} 1 & 2 \\ 2 & 4 \\ 3 & 6 \end{bmatrix} \in \mathbb{R}^{3 \times 2}$$

Column 2 = $2 \times$ column 1, so:

- $\text{range}(A) = \text{span} \{(1, 2, 3)^T\}$ — a **line** in $\mathbb{R}^3$, not a plane
- $\text{rank}(A) = 1$
- $A\mathbf{x} = (x_1 + 2x_2)(1, 2, 3)^T$, so
 $\text{null}(A) = \text{span} \{(2, -1)^T\}$ — a line in $\mathbb{R}^2$
- Check: $\text{rank} + \dim \text{null} = 1 + 1 = 2 = n$ ✓

Geometric reading: A **collapses** the null-space line to $\mathbf{0}$ and maps all of $\mathbb{R}^2$ onto the **range line** in $\mathbb{R}^3$. Rank counts the dimensions that *survive*; $\dim \text{null}$ counts the dimensions that are *destroyed* — the fundamental theorem is bookkeeping: $\text{survived} + \text{destroyed} = n$.



Computing the Range: Just the Column Span

$$\text{range}(A) = \{A\mathbf{x} : \mathbf{x} \in \mathbb{R}^n\} = \text{span}\{\mathbf{a}_1, \dots, \mathbf{a}_n\} \quad (\text{the columns of } A)$$

because $A\mathbf{x} = x_1\mathbf{a}_1 + \dots + x_n\mathbf{a}_n$: every output is a combination of columns, and every combination of columns is an output.

So the columns always span the range. The only work is trimming to a basis — discarding columns that are combinations of the others.

Our example: $A = \begin{bmatrix} 1 & 2 \\ 2 & 4 \\ 3 & 6 \end{bmatrix}$: here $\mathbf{a}_2 = 2\mathbf{a}_1$, so column 2 adds nothing:

$$\text{range}(A) = \text{span}\{\mathbf{a}_1, \mathbf{a}_2\} = \text{span}\{(1, 2, 3)^T\}, \quad \text{rank}(A) = 1$$

How to trim in general?

- row reduction (fine for tiny exact examples; numerically meaningless for data — it asks “exactly zero?”)
- This course’s methods: QR factorization and the SVD produce an *orthonormal* basis of $\text{range}(A)$ directly — stably, and with a tolerance replacing “exactly.”

The Singular Value Decomposition

Motivation: Three Questions One Factorization Will Answer

Question 1 — Understanding. A matrix $A \in \mathbb{R}^{m \times n}$ is mn numbers. What does it *do*? Is there a description of its action so complete that every property (rank, norms, sensitivity, invertibility) can simply be read off?

Question 2 — Compression. A 4000×3000 photo is 12 million numbers; a Netflix users-by-movies matrix is billions. These objects are stored and transmitted constantly. When is a matrix “effectively smaller” than its dimensions — and what is the *best possible* compression at a given size?

Question 3 — Solving the unsolvable. Data-fitting produces systems $A\mathbf{x} = \mathbf{b}$ with more equations than unknowns (no solution) or nearly dependent columns (infinitely many, all unstable). What should “solve” even mean — and can it be computed reliably?

Today's claim: one factorization — the SVD — answers all three, *for every matrix, no exceptions*.

Motivation: Why Not Just Use Eigenvalues?

You know one “reveal the structure” factorization already: diagonalization, $A = X\Lambda X^{-1}$.

Three ways it falls short as a general-purpose tool:

- ❶ **It may not exist.** Only square matrices have eigenvalues — data matrices rarely are.
Even square ones can be defective: $\begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}$ has no eigenvector basis.
- ❷ **The basis can be terrible.** X is generally *not orthogonal*; for non-normal matrices its columns can be nearly parallel, and X^{-1} amplifies errors enormously.
- ❸ **Eigenvalues can lie about size.** $\begin{bmatrix} 0 & 1000 \\ 0 & 0 \end{bmatrix}$ has both eigenvalues $= 0$, yet maps $\mathbf{e}_2$ to length 1000. Eigenvalues describe *invariant* directions, not *how strongly* A acts.

The fix: allow *two different* orthonormal bases — one for inputs, one for outputs. Then diagonal form is achievable for every matrix, with perfectly conditioned basis changes. That is the SVD: $A = U\Sigma V^T$.

(Eigenvalues remain the right tool for *dynamics* — powers and exponentials of A — Weeks 10–11.)

The SVD: Statement

Theorem (SVD)

Every $A \in \mathbb{R}^{m \times n}$ has a factorization

$$A = U\Sigma V^T \quad (23)$$

where $U \in \mathbb{R}^{m \times m}$ and $V \in \mathbb{R}^{n \times n}$ are orthogonal and $\Sigma \in \mathbb{R}^{m \times n}$ is diagonal with entries

$$\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_{\min(m,n)} \geq 0. \quad (24)$$

The σ_i are the **singular values**; the columns $\mathbf{u}_i$ of U and $\mathbf{v}_i$ of V are the left and right **singular vectors**.

Column-by-column form: $A\mathbf{v}_i = \sigma_i\mathbf{u}_i$ for $i = 1, \dots, \min(m, n)$.

The singular values are uniquely determined by A ; singular vectors are unique up to signs when the σ_i are distinct.

Example: SVD

Compute the SVD of $A = \begin{bmatrix} 3 & 0 \\ 4 & 5 \end{bmatrix}$ by hand.

Step 1: form $A^T A$ (we will justify this on the eigenvalue-connection slide):

$$A^T A = \begin{bmatrix} 3 & 4 \\ 0 & 5 \end{bmatrix} \begin{bmatrix} 3 & 0 \\ 4 & 5 \end{bmatrix} = \begin{bmatrix} 25 & 20 \\ 20 & 25 \end{bmatrix} \quad (25)$$

Step 2: eigenvalues of $A^T A$. Characteristic polynomial: $(25 - \lambda)^2 - 400 = 0$, so $25 - \lambda = \pm 20$:

$$\lambda_1 = 45, \quad \lambda_2 = 5 \quad \implies \quad \sigma_1 = \sqrt{45} = 3\sqrt{5} \approx 6.708, \quad \sigma_2 = \sqrt{5} \approx 2.236 \quad (26)$$

Step 3: eigenvectors of $A^T A$ = right singular vectors:

$$\mathbf{v}_1 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \quad \mathbf{v}_2 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ -1 \end{bmatrix} \quad (27)$$

Note $\sigma_1 = \sqrt{45} = \|A\|_2$ — the value we found by trial on Tuesday, now with a certificate.

Example: SVD

Step 4: left singular vectors from $A\mathbf{v}_i = \sigma_i \mathbf{u}_i$:

$$\mathbf{u}_1 = \frac{A\mathbf{v}_1}{\sigma_1} = \frac{1}{3\sqrt{5}} \cdot \frac{1}{\sqrt{2}} \begin{bmatrix} 3 \\ 9 \end{bmatrix} = \frac{1}{\sqrt{10}} \begin{bmatrix} 1 \\ 3 \end{bmatrix}, \quad \mathbf{u}_2 = \frac{A\mathbf{v}_2}{\sigma_2} = \frac{1}{\sqrt{10}} \begin{bmatrix} 3 \\ -1 \end{bmatrix} \quad (28)$$

Check: $\mathbf{u}_1^T \mathbf{u}_2 = \frac{3-3}{10} = 0 \checkmark$ (orthogonality is automatic — prove it!)

Step 5: assemble.

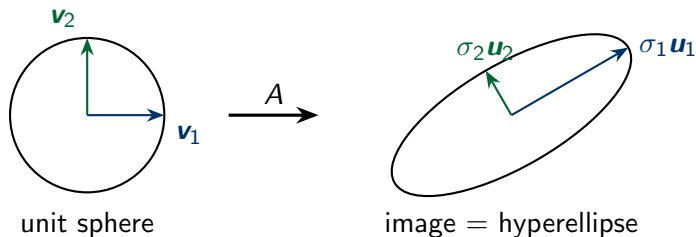
$$A = U\Sigma V^T = \frac{1}{\sqrt{10}} \begin{bmatrix} 1 & 3 \\ 3 & -1 \end{bmatrix} \begin{bmatrix} 3\sqrt{5} & 0 \\ 0 & \sqrt{5} \end{bmatrix} \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \quad (29)$$

Instant payoffs, read off the SVD:

- $\|A\|_2 = 3\sqrt{5} \approx 6.708$, $\|A\|_F = \sqrt{45+5} = \sqrt{50}$ (matches Tuesday's direct computation $\checkmark$)
- condition number $\kappa_2(A) = \sigma_1/\sigma_2 = 3$ (Week 6 preview)
- $|\det A| = \sigma_1\sigma_2 = 15$ (indeed $\det A = 15 \checkmark$)

Geometric Meaning: Hyperellipses

The image of the unit sphere under any matrix is a hyperellipse.



Reading $A = U\Sigma V^T$ right to left: rotate (V^T), stretch along axes (Σ), rotate (U).

- σ_i : lengths of the principal semi-axes
- u_i : directions of those semi-axes; v_i : their preimages

Existence: Proof Sketch

Idea: peel off one direction at a time.

- 1 Let $\sigma_1 = \|A\|_2 = \max_{\|\mathbf{v}\|_2=1} \|A\mathbf{v}\|_2$. By compactness the max is attained at some unit $\mathbf{v}_1$; set $\mathbf{u}_1 = A\mathbf{v}_1/\sigma_1$.
- 2 Extend to orthonormal bases: $V_1 = [\mathbf{v}_1 \ \tilde{V}]$, $U_1 = [\mathbf{u}_1 \ \tilde{U}]$. Then

$$U_1^T A V_1 = \begin{bmatrix} \sigma_1 & \mathbf{w}^T \\ \mathbf{0} & B \end{bmatrix} \quad (30)$$

- 3 Show $\mathbf{w} = \mathbf{0}$: otherwise the vector $(\sigma_1, \mathbf{w})^T$ would be stretched by more than σ_1 , contradicting maximality.
- 4 Recurse on the smaller matrix B . ■

Note the pattern: the top singular direction solves an optimization problem. This variational view returns next week (Courant–Fischer).

Reduced SVD and Rank-One Expansion

If $\text{rank}(A) = r$, only r singular values are nonzero. The **reduced (thin) SVD** keeps only what matters:

$$A = \hat{U} \hat{\Sigma} \hat{V}^T, \quad \hat{U} \in \mathbb{R}^{m \times r}, \quad \hat{\Sigma} \in \mathbb{R}^{r \times r}, \quad \hat{V} \in \mathbb{R}^{n \times r} \quad (31)$$

Equivalently, a sum of rank-one matrices:

$$A = \sum_{i=1}^r \sigma_i \mathbf{u}_i \mathbf{v}_i^T \quad (32)$$

ordered by importance: $\sigma_1 \geq \sigma_2 \geq \dots$

Everything is visible in the SVD:

- $\text{rank}(A) = r = \text{number of nonzero } \sigma_i$
- $\text{range}(A) = \text{span}\{\mathbf{u}_1, \dots, \mathbf{u}_r\}$, $\text{null}(A) = \text{span}\{\mathbf{v}_{r+1}, \dots, \mathbf{v}_n\}$
- $\|A\|_2 = \sigma_1$, $\|A\|_F = (\sigma_1^2 + \dots + \sigma_r^2)^{1/2}$

Example: The Rank-One Expansion, Explicitly

For our running example, $A = \sigma_1 \mathbf{u}_1 \mathbf{v}_1^T + \sigma_2 \mathbf{u}_2 \mathbf{v}_2^T$ works out to:

$$\sigma_1 \mathbf{u}_1 \mathbf{v}_1^T = 3\sqrt{5} \cdot \frac{1}{\sqrt{10}} \begin{bmatrix} 1 \\ 3 \end{bmatrix} \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \end{bmatrix} = \frac{3}{2} \begin{bmatrix} 1 & 1 \\ 3 & 3 \end{bmatrix} = \begin{bmatrix} 1.5 & 1.5 \\ 4.5 & 4.5 \end{bmatrix} \quad (33)$$

$$\sigma_2 \mathbf{u}_2 \mathbf{v}_2^T = \sqrt{5} \cdot \frac{1}{\sqrt{10}} \begin{bmatrix} 3 \\ -1 \end{bmatrix} \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & -1 \end{bmatrix} = \frac{1}{2} \begin{bmatrix} 3 & -3 \\ -1 & 1 \end{bmatrix} = \begin{bmatrix} 1.5 & -1.5 \\ -0.5 & 0.5 \end{bmatrix} \quad (34)$$

Sum:

$$\begin{bmatrix} 1.5 & 1.5 \\ 4.5 & 4.5 \end{bmatrix} + \begin{bmatrix} 1.5 & -1.5 \\ -0.5 & 0.5 \end{bmatrix} = \begin{bmatrix} 3 & 0 \\ 4 & 5 \end{bmatrix} = A \quad \checkmark \quad (35)$$

Notice: the first term already carries most of A — its Frobenius norm is $\sqrt{45} \approx 6.7$ of A 's total $\sqrt{50} \approx 7.07$, i.e., 90% of the “energy” $\sigma_1^2/(\sigma_1^2 + \sigma_2^2) = 45/50$.

Connection to Eigenvalue Problems

From $A = U\Sigma V^T$:

$$A^T A = V(\Sigma^T \Sigma)V^T, \quad AA^T = U(\Sigma \Sigma^T)U^T \quad (36)$$

Therefore:

- σ_i^2 = eigenvalues of $A^T A$ (and of AA^T)
- $\mathbf{v}_i$ = eigenvectors of $A^T A$; $\mathbf{u}_i$ = eigenvectors of AA^T

Warnings:

- This gives existence and intuition, **not an algorithm**: forming $A^T A$ squares the condition number (Week 5–6)
- Singular values $\neq$ eigenvalues in general! For symmetric A : $\sigma_i = |\lambda_i|$. For nonsymmetric matrices they can be wildly different.

How the SVD is computed: Week 11 (via the QR algorithm).

Low-Rank Approximation: The Eckart–Young Theorem

Truncate the rank-one expansion after k terms:

$$A_k = \sum_{i=1}^k \sigma_i \mathbf{u}_i \mathbf{v}_i^T \quad (37)$$

Theorem (Eckart–Young–Mirsky)

A_k is the **best rank- k approximation** of A in both norms:

$$\min_{\text{rank}(B) \leq k} \|A - B\|_2 = \|A - A_k\|_2 = \sigma_{k+1} \quad (38)$$

$$\min_{\text{rank}(B) \leq k} \|A - B\|_F = \|A - A_k\|_F = \left(\sum_{i>k} \sigma_i^2 \right)^{1/2} \quad (39)$$

Interpretation: the singular value tail tells you exactly how compressible a matrix is. Fast singular value decay $\Rightarrow$ the matrix is “approximately low-dimensional.”

Worked Example: Best Rank-One Approximation

For the running example, Eckart–Young says the best rank-1 approximation is the first term of the expansion:

$$A_1 = \sigma_1 \mathbf{u}_1 \mathbf{v}_1^T = \begin{bmatrix} 1.5 & 1.5 \\ 4.5 & 4.5 \end{bmatrix} \quad (40)$$

Error, both norms, no computation needed:

$$\|A - A_1\|_2 = \sigma_2 = \sqrt{5} \approx 2.236, \quad \|A - A_1\|_F = \sigma_2 = \sqrt{5} \quad (41)$$

(they coincide here because the remainder $\sigma_2 \mathbf{u}_2 \mathbf{v}_2^T$ has rank one).

The claim that should surprise you: no rank-1 matrix in the world — over all ∞^4 choices of $B = \mathbf{p}\mathbf{q}^T$ — gets closer to A than A_1 does.

Quick contrast: the “obvious” rank-1 guess, keeping A ’s dominant column pattern, e.g.

$B = \begin{bmatrix} 3 & 0 \\ 4 & 0 \end{bmatrix}$, gives $\|A - B\|_F = 5 > \sqrt{5}$. Optimality is not obvious — it is a theorem.

Why Low-Rank Approximation Runs the Modern World

Storage: A_k needs $k(m + n + 1)$ numbers instead of mn .

For a 1000×1000 image at $k = 50$: about 10% of the original storage.

Applications of the same theorem:

- **Image compression** — this week's lab
- **PCA / dimension reduction** — next Tuesday
- **Recommender systems** — users $\times$ items matrices are approximately low rank
- **Model compression & LoRA** — fine-tuning updates to neural network weight matrices are well approximated at very low rank (next Thursday's spotlight)
- **Reduced-order modeling** — simulation snapshot matrices in fluid dynamics have rapidly decaying singular values (next Thursday's scientific computing connection)

Numerical Preview: Singular Value Spectra in the Wild

What you will see in Lab 1 (representative values, 512×512 matrices):

A natural photograph:

$$\sigma_1 \approx 6.1 \times 10^4, \quad \sigma_{10} \approx 4.2 \times 10^3, \quad \sigma_{50} \approx 6.0 \times 10^2, \quad \sigma_{200} \approx 7.5 \times 10^1 \quad (42)$$

Roughly geometric decay over three orders of magnitude: $\sigma_{50}/\sigma_1 \approx 10^{-2}$. Rank 50 ($\approx 20\%$ storage) already looks sharp.

An i.i.d. Gaussian noise matrix (`numpy.random.randn`):

$$\sigma_1 \approx 45.2, \quad \sigma_{100} \approx 38.6, \quad \sigma_{300} \approx 24.1, \quad \sigma_{512} \approx 0.4 \quad (43)$$

Nearly flat: $\sigma_{100}/\sigma_1 \approx 0.85$. **Noise is incompressible** — random matrix theory (the Marchenko–Pastur law) says the singular values fill a bulk with no gaps.

Why the difference? Images have *structure*: smooth regions, repeated patterns, long-range correlations — so columns are nearly linearly dependent. Noise, by definition, has none.

This contrast is the entire basis of denoising by truncated SVD: signal concentrates in few directions, noise spreads over all of them.

Lab 1: SVD Image Compression

This week's lab (Jupyter notebook, NumPy + Matplotlib):

- 1 Load a grayscale image as a matrix $A \in \mathbb{R}^{m \times n}$
- 2 Compute the SVD: `U, s, Vt = numpy.linalg.svd(A, full_matrices=False)`
- 3 Form A_k for $k = 1, 5, 20, 50, 100$ and display the reconstructions
- 4 Plot the singular value spectrum σ_i vs. i (log scale)
- 5 Plot relative error $\|A - A_k\|_F / \|A\|_F$ vs. k and verify Eckart–Young: compare against $(\sum_{i>k} \sigma_i^2)^{1/2} / \|A\|_F$
- 6 Compute the compression ratio at each k

Question to think about: why do natural images have decaying singular values, while a matrix of pure noise does not? Try it: repeat with `numpy.random.randn(m, n)`.

Summary

- Orthogonal matrices preserve geometry and rounding behavior — they are the building blocks of stable algorithms
- The SVD $A = U\Sigma V^T$ exists for every matrix and reveals rank, norms, and the four fundamental subspaces
- Eckart–Young: truncating the SVD gives the *optimal* low-rank approximation, with error σ_{k+1}